

HuggingFace를 활용한

인공지능 웹 애플리케이션 개발

제 6장

컴퓨터 비전 웹 서비스

이 장의 학습 목표

이미지 분류 모델(ResNet, ViT)을 pipeline으로 래핑하여 업로드 기반 분류 API를 구현합니다.

YOLOv8로 객체 탐지 API를 만들고 바운딩 박스 시각화 이미지를 반환합니다.

BLIP 모델로 이미지 내용을 자연어로 설명하는 이미지 캡셔닝 API를 구현합니다.

Jinja2 템플릿으로 세 가지 CV 서비스를 하나의 웹 UI로 통합하여 완성합니다.

목 차

제6장 컴퓨터 비전 웹 서비스

★ 이 장의 개요

이 장에서는 Hugging Face와 외부 CV 모델을 활용해 이미지를 분석하고 처리하는 AI 웹 서비스를 FastAPI 기반으로 구축합니다. 이미지 분류, 객체 탐지, 이미지 설명 생성 API를 차례로 구현하고, 마지막으로 Jinja2 템플릿을 이용해 세 서비스를 하나의 웹 UI로 통합합니다.

이 장을 마치면 이미지 파일 업로드 → AI 추론 → 결과 반환의 전 과정을 경험하게 됩니다.

컴퓨터 비전(Computer Vision, CV)은 컴퓨터가 이미지와 영상에서 의미 있는 정보를 인식하고 이해하는 AI 분야입니다. 스마트폰 얼굴 인식, 자율주행 차량 장애물 감지, 의료 영상 진단, 보안 CCTV 분석 등 수많은 현실 서비스의 핵심 기술입니다.

Hugging Face는 ResNet, ViT, ConvNeXt 등 이미지 분류 모델과 BLIP, GIT 등 멀티모달 모델을 Pipeline API로 통합 제공하며, ultralytics의 YOLOv8은 객체 탐지 분야의 사실상 표준 모델입니다. 이 두 생태계를 FastAPI로 결합하면 엔터프라이즈 수준의 CV 웹 서비스를 빠르게 구축할 수 있습니다.

📁 6장 프로젝트 디렉터리 구조

chapter6- cv- services/

```

|—— main.py                # FastAPI 앱 진입점
|—— requirements.txt
|—— .env
|—— routers/
|   |—— classification.py    # 6.1 이미지 분류 라우터
|   |—— detection.py        # 6.2 객체 탐지 라우터
|   |—— captioning.py       # 6.3 이미지 캡셔닝 라우터
|   |—— ui.py               # 6.4 Jinja2 웹 UI 라우터
|—— services/
|   |—— classification_service.py

```

```

|   ├── detection_service.py
|   ├── captioning_service.py
|   └── image_utils.py
├── models/
|   └── schemas.py
├── templates/
|   ├── base.html          # 공통 레이아웃
|   ├── index.html         # 메인 페이지
|   ├── classify.html       # 이미지 분류 UI
|   ├── detect.html        # 객체 탐지 UI
|   └── caption.html        # 이미지 캡셔닝 UI
├── static/
|   ├── css/style.css
|   └── js/app.js
├── temp/
└── output/

```

requirements.txt

```

# Web Framework
fastapi==0.111.0
uvicorn[standard]==0.29.0
python-multipart==0.0.9
aiofiles==23.2.1
jinja2==3.1.4          # 템플릿 엔진

# CV / AI
torch==2.3.0
torchvision==0.18.0
transformers==4.41.0
ultralytics==8.2.18    # YOLOv8

# 이미지 처리
Pillow==10.3.0

```

```
opencv-python==4.9.0.80
```

```
numpy==1.26.4
```

```
# 유틸리티
```

```
pydantic==2.7.1
```

```
python-dotenv==1.0.1
```

```
httpx==0.27.0
```

6.1 이미지 분류 모델로 웹 앱 만들기

이미지 분류(Image Classification)는 입력된 이미지가 어떤 카테고리에 속하는지를 판별하는 가장 기본적인 컴퓨터 비전 태스크입니다. '이 사진은 고양이입니다', '이 X-Ray 이미지에는 폐렴 소견이 있습니다' 등의 판별이 이에 해당합니다.

6.1.1 이미지 분류란? (ResNet, ViT, ConvNeXt 등 모델 구조 개요)

이미지 분류 모델의 역사는 CNN(Convolutional Neural Network)에서 시작합니다. 2012년 AlexNet이 ImageNet 대회에서 압도적인 성능을 보인 이후, 더 깊고 정확한 CNN 모델들이 개발되었습니다. 2020년대에는 NLP에서 성공한 Transformer 구조가 이미지 분야로 이전되어 ViT(Vision Transformer) 계열이 CNN을 능가하기 시작했습니다.

모델	아키텍처	ImageNet Top-1	특징
ResNet-50	CNN (잔차 연결)	76.1%	경량·빠름·범용. 전이학습 베이스라
EfficientNet-B4	CNN (NAS 최적화)	82.9%	파라미터 효율 최고. 모바일 배포용
ViT-Base/16	Pure Transformer	81.8%	패치 기반 자기 어텐션. 대규모 데이터 필요.
DeiT-Base	Distilled ViT	81.8%	지식 증류로 소규모 데이터에서도 학습 가능.
ConvNeXt-Base	CNN+Transformer	83.8%	CNN 구조에 Transformer 기법 적
Swin-Transformer-B	Hierarchical ViT	83.5%	윈도우 어텐션. 밀집 예측 태스크에

이미지 분류의 핵심 처리 파이프라인을 단계별로 살펴봅니다.

1. 이미지 로드: PIL 또는 OpenCV로 이미지 읽기
2. 전처리: 크기 조정(224× 224 등), 정규화 (채널 평균/표준편차 기준)
3. 텐서 변환: numpy 배열 → PyTorch Tensor, [H,W,C] → [B,C,H,W]
4. 모델 추론: model(tensor) → 클래스 점수 벡터 (logits)
5. 후처리: Softmax → 확률값 → Top-K 클래스 추출
6. 결과 반환: 클래스 레이블, 신뢰도 점수 JSON 반환

6.1.2 Hugging Face pipeline(task='image-classification') 실습

Hugging Face Pipeline API는 이미지 분류의 전처리~후처리를 모두 자동화해줍니다. 단 몇 줄의 코드로 최신 이미지 분류 모델을 즉시 사용할 수 있습니다.

services/classification_service.py

```
# services/classification_service.py
import io, time, torch, base64
from typing import List, Dict, Optional
from PIL import Image
from transformers import pipeline

class ImageClassificationService:
    SUPPORTED_MODELS = {
        'vit-base': 'google/vit-base-patch16-224',
        'convnext-base': 'facebook/convnext-base-224-22k-1k',
        'resnet-50': 'microsoft/resnet-50',
        'efficientnet': 'google/efficientnet-b4',
        'deit-base': 'facebook/deit-base-distilled-patch16-224',
    }

    def __init__(self, model_key: str = 'vit-base'):
        self.model_key = model_key
        self.pipe = None
        self.device = 0 if torch.cuda.is_available() else -1

    def load_model(self, model_key: Optional[str] = None):
        key = model_key or self.model_key
        model_id = self.SUPPORTED_MODELS.get(key, self.SUPPORTED_MODELS['vit-base'])
        print(f'[Classification] Loading: {model_id}')
        self.pipe = pipeline(
            task='image-classification',
            model=model_id,
            device=self.device,
        )
```

```
self.model_key = key

def classify_from_file(self, image_bytes: bytes, top_k: int = 5) -> Dict:
    if self.pipe is None:
        raise RuntimeError('분류 모델이 로드되지 않았습니다.')
    start = time.time()
    image = Image.open(io.BytesIO(image_bytes)).convert('RGB')
    results = self.pipe(image, top_k=top_k)
    elapsed = round(time.time() - start, 3)
    return {
        'predictions': [
            {'rank': i+1, 'label': r['label'],
            'score': round(r['score'],6),
            'percentage': round(r['score']*100,2)}
            for i, r in enumerate(results)
        ],
        'top_prediction': results[0]['label'],
        'top_score': round(results[0]['score'],6),
        'model_used': self.SUPPORTED_MODELS[self.model_key],
        'inference_time_sec': elapsed,
        'image_size': f'{image.width}x{image.height}',
    }

def classify_from_base64(self, b64_string: str, top_k: int = 5) -> Dict:
    if ',' in b64_string:
        b64_string = b64_string.split(',', 1)[1]
    return self.classify_from_file(base64.b64decode(b64_string), top_k)

def classify_from_url(self, url: str, top_k: int = 5) -> Dict:
    import httpx
    resp = httpx.get(url, timeout=10)
    resp.raise_for_status()
    return self.classify_from_file(resp.content, top_k)

classification_service = ImageClassificationService(model_key='vit-base')
```


6.1.3 UploadFile로 이미지 업로드 받고 분류 결과 반환

💡 TIP: Base64 인코딩 이미지를 JSON으로 받는 방법

파일 업로드(multipart/form-data) 대신 Base64 인코딩으로 이미지를 JSON Body에 포함시키면 단일 JSON 요청으로 이미지와 메타데이터를 함께 전송할 수 있습니다.

Python에서 이미지를 Base64로 변환:

```
import base64
with open('image.jpg', 'rb') as f:
    b64 = base64.b64encode(f.read()).decode()
payload = {'image_base64': f'data:image/jpeg;base64,{b64}'}
```

단점: 파일 크기가 약 33% 증가합니다. 10MB 이미지 → 약 13.3MB JSON.

대용량 이미지는 UploadFile 방식을 권장합니다.

models/schemas.py - 이미지 분류 스키마

```
# models/schemas.py
from pydantic import BaseModel, Field
from typing import List, Dict, Optional

class PredictionItem(BaseModel):
    rank: int
    label: str
    score: float
    percentage: float

class ClassificationResponse(BaseModel):
    filename: str
    predictions: List[PredictionItem]
    top_prediction: str
    top_score: float
    model_used: str
    inference_time_sec: float
    image_size: str
```

```

class Base64ClassifyRequest(BaseModel):
    image_base64: str = Field(..., description='Base64 인코딩 이미지')
    top_k: int = Field(5, ge=1, le=20)

    model_config = {
        'json_schema_extra': {
            'examples': [{'image_base64': 'data:image/jpeg;base64,...', 'top_k': 5}]
        }
    }

```

routers/classification.py

```

# routers/classification.py
from fastapi import APIRouter, UploadFile, File, Form, HTTPException
from typing import Optional
from models.schemas import ClassificationResponse, Base64ClassifyRequest
from services.classification_service import classification_service

router = APIRouter(prefix='/classify', tags=['Image Classification'])

SUPPORTED_FORMATS = {'jpg', 'jpeg', 'png', 'bmp', 'webp', 'tiff'}

def _validate(filename, size):
    from pathlib import Path
    ext = Path(filename).suffix.lower()
    if ext not in SUPPORTED_FORMATS:
        raise ValueError(f'지원하지 않는 형식: {ext}')
    if size > 10*1024*1024:
        raise ValueError('10MB 초과 파일은 업로드 불가')

@router.post('/upload', response_model=ClassificationResponse,
             summary='이미지 파일 업로드 후 분류')
async def classify_upload(
    file: UploadFile = File(..., description='분류할 이미지 (JPEG/PNG/WEBP)'),
    top_k: int = Form(5, ge=1, le=20),
    model_key: str = Form('vit-base'),
):

```

```

content = await file.read()

try:
    _validate(file.filename, len(content))
except ValueError as e:
    raise HTTPException(422, str(e))

if model_key != classification_service.model_key:
    try:
        classification_service.load_model(model_key)
    except Exception as e:
        raise HTTPException(400, f'모델 로드 실패: {e}')

    try:
        result = classification_service.classify_from_file(content, top_k=top_k)
    except Exception as e:
        raise HTTPException(500, f'분류 오류: {e}')

    return ClassificationResponse(filename=file.filename, **result)

@router.post('/base64', response_model=ClassificationResponse,
             summary='Base64 이미지 분류')
async def classify_base64(request: Base64ClassifyRequest):
    try:
        result = classification_service.classify_from_base64(
            request.image_base64, top_k=request.top_k
        )
    except Exception as e:
        raise HTTPException(500, f'Base64 분류 오류: {e}')

    return ClassificationResponse(filename='base64_image', **result)

@router.post('/url', response_model=ClassificationResponse,
             summary='URL 이미지 분류')
async def classify_url(url: str, top_k: int = 5):
    try:
        result = classification_service.classify_from_url(url, top_k=top_k)
    except Exception as e:
        raise HTTPException(500, f'URL 분류 오류: {e}')

    return ClassificationResponse(filename=url, **result)

@router.get('/models', summary='지원 모델 목록')

```

```
async def list_models():  
    return {  
        'models': classification_service.SUPPORTED_MODELS,  
        'current': classification_service.model_key,  
        'loaded': classification_service.pipe is not None,  
    }
```

분류 API 테스트

```
$ curl -X POST http://localhost:8000/classify/upload ₩  
  -F 'file=@dog.jpg' -F 'top_k=5' -F 'model_key=vit-base'
```

응답 예시:

```
{  
  "filename": "dog.jpg",  
  "predictions": [  
    {"rank": 1, "label": "golden retriever", "score": 0.9432, "percentage": 94.32},  
    {"rank": 2, "label": "Labrador retriever", "score": 0.0341, "percentage": 3.41}  
  ],  
  "top_prediction": "golden retriever",  
  "top_score": 0.9432,  
  "model_used": "google/vit-base-patch16-224",  
  "inference_time_sec": 0.412,  
  "image_size": "640x480"  
}
```

6.2 객체 탐지 API 만들기 (YOLOv8)

객체 탐지(Object Detection)는 이미지 내에서 여러 객체의 위치(Bounding Box)와 클래스(종류)를 동시에 예측하는 태스크입니다. 단순히 '이미지에 무엇이 있는가'를 넘어 '어디에 있는가'까지 알려주는 더 복잡한 비전 태스크입니다.

6.2.1 객체 탐지란? 이미지 속 객체 위치(Box) + 클래스 예측

객체 탐지 결과는 탐지된 각 객체에 대해 바운딩 박스 좌표 $[x1, y1, x2, y2]$, 클래스 이름, 신뢰도 점수를 함께 반환합니다. IoU(Intersection over Union)와 NMS(Non-Maximum Suppression)가 핵심 알고리즘입니다.

탐지 방식	대표 모델	특징
1-Stage Detector	YOLO, SSD, RetinaNet	단일 네트워크로 박스+클래스 동시 예측. 실시간 처리 적합.
2-Stage Detector	Faster R-CNN, Mask R-CNN	후보 영역 추출 후 분류. 정확도 높음. 느림.
Anchor-Free	FCOS, CenterNet	앵커 박스 없이 키포인트 기반. 최신 트렌드.
Transformer 기반	DETR, Deformable DETR	End-to-End 탐지. NMS 불필요.

6.2.2 YOLOv8 개요 및 설치 (ultralytics 라이브러리 사용)

YOLOv8은 Ultralytics사가 2023년 발표한 최신 YOLO 시리즈 모델입니다. 객체 탐지뿐만 아니라 세그멘테이션, 포즈 추정, 분류, OBB 등 다양한 CV 태스크를 단일 프레임워크로 지원합니다.

모델	파라미터	mAP50-95	속도 및 적합 환경
YOLOv8n	3.2M	37.3	초고속(80+FPS). 모바일/엣지 기기용.
YOLOv8s	11.2M	44.9	빠름. 경량 서버/API에 적합.
YOLOv8m	25.9M	50.2	속도-정확도 균형. API 서비스 권장.
YOLOv8l	43.7M	52.9	높은 정확도. GPU 권장.

모델	파라미터	mAP50-95	속도 및 적합 환경
YOLOv8x	68.2M	53.9	최고 정확도. 고사양 GPU 필요.

```
# YOLOv8 설치
```

```
$ pip install ultralytics
```

```
# 설치 확인 및 첫 번째 추론
```

```
$ python -c "from ultralytics import YOLO; m=YOLO('yolov8n.pt'); print(m.info())"
```

```
# YOLO CLI 빠른 테스트
```

```
$ yolo predict model=yolov8n.pt source='https://ultralytics.com/images/bus.jpg'
```

```
# Python API 기본 사용법
```

```
from ultralytics import YOLO
```

```
model = YOLO('yolov8n.pt') # 모델 로드 (없으면 자동 다운로드)
```

```
results = model.predict('image.jpg') # 추론
```

```
for r in results:
```

```
    print(r.boxes.xyxy) # 바운딩 박스 좌표
```

```
    print(r.boxes.conf) # 신뢰도
```

```
    print(r.boxes.cls) # 클래스 ID
```

6.2.3 model.predict(source=image)로 이미지 추론

services/detection_service.py

```
# services/detection_service.py
```

```
import io, time
```

```
from PIL import Image, ImageDraw
```

```
from typing import List, Dict, Optional
```

```
try:
```

```
    from ultralytics import YOLO
```

```
    YOLO_AVAILABLE = True
```

```
except ImportError:
```

```
    YOLO_AVAILABLE = False
```

```

COCO_COLORS = [
    (255,0,0),(0,255,0),(0,0,255),(255,255,0),
    (255,0,255),(0,255,255),(128,0,0),(0,128,0),
    (0,0,128),(128,128,0),(128,0,128),(0,128,128),
]

class YOLODetectionService:
    SUPPORTED_MODELS = {
        'yolov8n': 'yolov8n.pt',
        'yolov8s': 'yolov8s.pt',
        'yolov8m': 'yolov8m.pt',
        'yolov8l': 'yolov8l.pt',
    }

    def __init__(self, model_key: str = 'yolov8n'):
        self.model_key = model_key
        self.model = None

    def load_model(self, model_key: Optional[str] = None):
        if not YOLO_AVAILABLE:
            raise RuntimeError('ultralytics 패키지 미설치')
        key = model_key or self.model_key
        model_file = self.SUPPORTED_MODELS.get(key, 'yolov8n.pt')
        print(f'[Detection] Loading YOLOv8: {model_file}')
        self.model = YOLO(model_file)
        self.model_key = key

    def detect(
        self, image_bytes: bytes,
        confidence: float = 0.25,
        iou_threshold: float = 0.7,
        max_det: int = 300,
    ) -> Dict:
        if self.model is None:
            raise RuntimeError('탐지 모델 미로드')
        start = time.time()
        pil_image = Image.open(io.BytesIO(image_bytes)).convert('RGB')

```

```

img_w, img_h = pil_image.size
results = self.model.predict(
    source=pil_image, conf=confidence,
    iou=iou_threshold, max_det=max_det, verbose=False,
)
elapsed = round(time.time() - start, 3)
detections = []
result = results[0]
if result.bboxes is not None and len(result.bboxes) > 0:
    boxes = result.bboxes
    for i in range(len(boxes)):
        x1,y1,x2,y2 = boxes.xyxy[i].tolist()
        conf = float(boxes.conf[i])
        cls_id = int(boxes.cls[i])
        cls_name = self.model.names[cls_id]
        detections.append({
            'id': i+1,
            'class_id': cls_id,
            'class_name': cls_name,
            'confidence': round(conf, 4),
            'bbox': {'x1':round(x1),'y1':round(y1),
                    'x2':round(x2),'y2':round(y2),
                    'width':round(x2-x1),'height':round(y2-y1)},
            'bbox_normalized': {
                'x1':round(x1/img_w,4),'y1':round(y1/img_h,4),
                'x2':round(x2/img_w,4),'y2':round(y2/img_h,4),
            },
        })
class_counts = {}
for d in detections:
    class_counts[d['class_name']] = class_counts.get(d['class_name'],0)+1
return {
    'detections': detections,
    'total_objects': len(detections),
    'class_counts': class_counts,
    'image_size': f'{img_w}x{img_h}',
    'model_used': self.model_key,
    'inference_time_sec': elapsed,

```



```

    }

def draw_boxes(self, image_bytes: bytes, detections: List[Dict]) -> bytes:
    pil_image = Image.open(io.BytesIO(image_bytes)).convert('RGB')
    draw = ImageDraw.Draw(pil_image)
    for det in detections:
        bbox = det['bbox']
        x1,y1,x2,y2 = bbox['x1'],bbox['y1'],bbox['x2'],bbox['y2']
        color = COCO_COLORS[det['class_id'] % len(COCO_COLORS)]
        for t in range(2):
            draw.rectangle([x1+t,y1+t,x2-t,y2-t], outline=color)
        label = f'{det["class_name"]} {det["confidence"]:.2f}'
        tb = draw.textbbox((x1,y1), label)
        tw,th = tb[2]-tb[0], tb[3]-tb[1]
        draw.rectangle([x1, y1-th-4, x1+tw+4, y1], fill=color)
        draw.text((x1+2, y1-th-2), label, fill=(255,255,255))
    buf = io.BytesIO()
    pil_image.save(buf, format='JPEG', quality=90)
    return buf.getvalue()

detection_service = YOLODetectionService(model_key='yolov8n')

```

6.2.4 FastAPI에서 이미지 업로드 → 예측 결과(JSON or 시각화 이미지) 반환

6.2.5 결과 시각화 이미지 저장 or base64로 응답

`routers/detection.py`

```

# routers/detection.py
import base64

from fastapi import APIRouter, UploadFile, File, Form, HTTPException
from fastapi.responses import Response
from pathlib import Path
from typing import Optional, List
from pydantic import BaseModel
from services.detection_service import detection_service

router = APIRouter(prefix='/detect', tags=['Object Detection (YOLOv8)'])

```

```
class BoundingBox(BaseModel):
    x1: int; y1: int; x2: int; y2: int
    width: int; height: int

class DetectionItem(BaseModel):
    id: int
    class_id: int
    class_name: str
    confidence: float
    bbox: BoundingBox

class DetectionResponse(BaseModel):
    filename: str
    detections: List[DetectionItem]
    total_objects: int
    class_counts: dict
    image_size: str
    model_used: str
    inference_time_sec: float
    visualization_base64: Optional[str] = None

@router.post('/json', response_model=DetectionResponse,
             summary='객체 탐지 JSON 결과 반환')
async def detect_json(
    file: UploadFile = File(...),
    confidence: float = Form(0.25, ge=0.01, le=0.99),
    iou_threshold: float = Form(0.7, ge=0.1, le=0.99),
    max_detections: int = Form(100, ge=1, le=500),
    include_visualization: bool = Form(False),
    model_key: str = Form('yolov8n'),
):
    content = await file.read()
    ext = Path(file.filename).suffix.lower()
    if ext not in {'jpg', 'jpeg', '.png', '.bmp', '.webp'}:
        raise HTTPException(422, f'지원하지 않는 형식: {ext}')
```

```

if model_key != detection_service.model_key:
    detection_service.load_model(model_key)

try:
    result = detection_service.detect(
        content, confidence=confidence,
        iou_threshold=iou_threshold, max_det=max_detections
    )
except Exception as e:
    raise HTTPException(500, f'탐지 오류: {e}')

viz_b64 = None
if include_visualization and result['detections']:
    viz_bytes = detection_service.draw_boxes(content, result['detections'])
    viz_b64 = 'data:image/jpeg;base64,' + base64.b64encode(viz_bytes).decode()

return DetectionResponse(
    filename=file.filename,
    visualization_base64=viz_b64,
    **result
)

@router.post('/visualize', summary='객체 탐지 시각화 이미지 반환',
    response_class=Response,
    responses={200: {'content': {'image/jpeg': {}}}})
async def detect_visualize(
    file: UploadFile = File(...),
    confidence: float = Form(0.25, ge=0.01, le=0.99),
):
    content = await file.read()
    result = detection_service.detect(content, confidence=confidence)
    if not result['detections']:
        return Response(content=content, media_type='image/jpeg')
    viz_bytes = detection_service.draw_boxes(content, result['detections'])
    return Response(

```

```

        content=viz_bytes, media_type='image/jpeg',
        headers={
            'X-Total-Objects': str(result['total_objects']),
            'X-Inference-Time': str(result['inference_time_sec']),
        },
    )

@router.get('/classes', summary='탐지 가능한 COCO 클래스 목록')
async def get_classes():
    if detection_service.model is None:
        raise HTTPException(503, '모델 미로드')
    return {'classes': detection_service.model.names,
            'total': len(detection_service.model.names)}

```

팁: Swagger에서 테스트 가능한 객체 탐지 API 만들기

Swagger UI(/docs)에서 객체 탐지 API를 테스트하는 방법:

1. POST /detect/json → Try it out 클릭
2. file 항목의 Choose File로 이미지 업로드
3. confidence: 0.4, include_visualization: true 설정
4. Execute → Response body에서 detections 배열 확인

시각화 이미지를 직접 다운로드하려면:

```
curl -X POST http://localhost:8000/detect/visualize \
-F 'file=@street.jpg' --output result.jpg
```

curl로 JSON 결과만 받으려면:

```
curl -X POST http://localhost:8000/detect/json \
-F 'file=@street.jpg' -F 'confidence=0.4'
```

6.3 이미지 설명 생성기 API (BLIP)

이미지 캡셔닝(Image Captioning)은 이미지를 입력받아 그 내용을 설명하는 자연어 문장을 생성하는 멀티모달 AI 태스크입니다. 시각 장애인을 위한 이미지 대체 텍스트 자동 생성, 이미지 검색 시스템의 메타데이터 생성, 미디어 자동 분류 등에 활용됩니다.

6.3.1 이미지 캡셔닝(Image Captioning)이란?

이미지 캡셔닝은 컴퓨터 비전과 자연어 처리가 결합된 멀티모달 태스크입니다. Vision Encoder로 이미지를 특징 벡터로 변환하고, 크로스 어텐션을 통해 언어 모델이 설명 문장을 자동 회귀 방식으로 생성합니다.

모델	아키텍처	특징 및 활용
BLIP	ViT + BERT 계열	이미지 캡셔닝, VQA, 검색. 경량. Salesforce 개발.
BLIP-2	ViT + Q-Former + LLM	LLM 연결로 강력한 추론. GPU 권장. 고질.
GIT (GenerativeIT)	ViT + GPT-2	Microsoft 개발. 이미지 캡셔닝에 특화.
InstructBLIP	BLIP-2 + Instruction	지시 기반 다양한 시각 추론.
LLaVA	CLIP + LLaMA/Mistral	강력한 오픈소스 멀티모달 LLM.

6.3.2 Salesforce/blip-image-captioning-base 모델 개요

BLIP(Bootstrapping Language-Image Pre-training)은 Salesforce Research가 2022년 발표한 멀티모달 모델입니다. 이미지-텍스트 쌍을 필터링하고 캡션 생성으로 품질을 향상시키는 Bootstrapping 전략이 핵심입니다.

BLIP 모델 변형	모델 크기 및 특징
Salesforce/blip-image-captioning-base	약 250M B. 기본 캡셔닝. CPU에서도 동작 가능.
Salesforce/blip-image-captioning-large	약 900M B. 더 풍부한 캡션. GPU 권장.
Salesforce/blip-vqa-base	Visual Question Answering용 파인튜닝 버전.

BLIP 모델 변형	모델 크기 및 특징
Salesforce/blip2-opt-2.7b	BLIP-2 + OPT 2.7B. 강력한 추론. 큰 메모리 필요.

6.3.3 사용자가 업로드한 이미지 → 자연어 문장 생성

6.3.4 transformers + vision transformers 연동 실습

6.3.5 BlipProcessor, BlipForConditionalGeneration 사용법

services/captioning_service.py

```
# services/captioning_service.py
import io, time, torch
from PIL import Image
from typing import Optional, Dict
from transformers import (
    BlipProcessor, BlipForConditionalGeneration,
    BlipForQuestionAnswering,
)

class BLIPCaptioningService:
    CAPTIONING_MODELS = {
        'base': 'Salesforce/blip-image-captioning-base',
        'large': 'Salesforce/blip-image-captioning-large',
    }
    VQA_MODEL = 'Salesforce/blip-vqa-base'

    def __init__(self, model_key: str = 'base'):
        self.model_key = model_key
        self.processor: Optional[BlipProcessor] = None
        self.caption_model: Optional[BlipForConditionalGeneration] = None
        self.vqa_processor: Optional[BlipProcessor] = None
        self.vqa_model: Optional[BlipForQuestionAnswering] = None
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

    def load_captioning_model(self, model_key: Optional[str] = None):
        key = model_key or self.model_key
```

```

model_id = self.CAPTIONING_MODELS.get(key, self.CAPTIONING_MODELS['base'])
print(f'[BLIP] Loading captioning model: {model_id}')
self.processor = BlipProcessor.from_pretrained(model_id)
self.caption_model = BlipForConditionalGeneration.from_pretrained(
    model_id,
    torch_dtype=torch.float16 if self.device.type=='cuda' else torch.float32,
).to(self.device)
self.caption_model.eval()
self.model_key = key

```

```

def load_vqa_model(self):
    print(f'[BLIP] Loading VQA model: {self.VQA_MODEL}')
    self.vqa_processor = BlipProcessor.from_pretrained(self.VQA_MODEL)
    self.vqa_model = BlipForQuestionAnswering.from_pretrained(
        self.VQA_MODEL
    ).to(self.device)
    self.vqa_model.eval()

```

```

def generate_caption(
    self, image_bytes: bytes,
    conditional_text: Optional[str] = None,
    max_new_tokens: int = 50,
    num_beams: int = 5,
    num_captions: int = 1,
) -> Dict:
    """
    이미지에 대한 자연어 설명을 생성합니다.
    conditional_text: 생성 방향을 유도하는 텍스트 (예: 'a photo of')
    """
    if self.caption_model is None:
        raise RuntimeError('캡셔닝 모델이 로드되지 않았습니다.')
    start = time.time()
    image = Image.open(io.BytesIO(image_bytes)).convert('RGB')

    if conditional_text:
        inputs = self.processor(
            images=image, text=conditional_text, return_tensors='pt',

```

```

        ).to(self.device)
    else:
        inputs = self.processor(
            images=image, return_tensors='pt',
        ).to(self.device)

    with torch.no_grad():
        output_ids = self.caption_model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            num_beams=num_beams,
            num_return_sequences=min(num_captions, num_beams),
            early_stopping=True,
            no_repeat_ngram_size=3,
        )

    captions = self.processor.batch_decode(output_ids, skip_special_tokens=True)
    elapsed = round(time.time() - start, 3)
    return {
        'captions': captions,
        'primary_caption': captions[0],
        'conditional_text': conditional_text,
        'image_size': f'{image.width}x{image.height}',
        'model_used': self.CAPTIONING_MODELS[self.model_key],
        'inference_time_sec': elapsed,
    }

def answer_question(self, image_bytes: bytes, question: str) -> Dict:
    if self.vqa_model is None:
        raise RuntimeError("VQA 모델이 로드되지 않았습니다.")
    start = time.time()
    image = Image.open(io.BytesIO(image_bytes)).convert('RGB')
    inputs = self.vqa_processor(
        images=image, text=question, return_tensors='pt',
    ).to(self.device)
    with torch.no_grad():
        out = self.vqa_model.generate(**inputs, max_new_tokens=50)

```



```

        answer = self.vqa_processor.decode(out[0], skip_special_tokens=True)

    return {'question': question, 'answer': answer,
            'inference_time_sec': round(time.time()-start, 3)}

captioning_service = BLIPCaptioningService(model_key='base')

```

routers/captioning.py

```

# routers/captioning.py
from fastapi import APIRouter, UploadFile, File, Form, HTTPException
from typing import Optional, List
from pydantic import BaseModel
from services.captioning_service import captioning_service

router = APIRouter(prefix='/caption', tags=['Image Captioning (BLIP)'])

class CaptionResponse(BaseModel):
    filename: str
    captions: List[str]
    primary_caption: str
    conditional_text: Optional[str]
    image_size: str
    model_used: str
    inference_time_sec: float

class VQAResponse(BaseModel):
    filename: str
    question: str
    answer: str
    inference_time_sec: float

@router.post('/generate', response_model=CaptionResponse,
             summary='이미지 캡션 생성')
async def generate_caption(
    file: UploadFile = File(...),
    conditional_text: Optional[str] = Form(None,
    description='캡션 유도 텍스트 (예: a photo of)'),

```

```

max_new_tokens: int = Form(50, ge=10, le=200),
num_beams: int = Form(5, ge=1, le=10),
num_captions: int = Form(1, ge=1, le=5),
model_key: str = Form('base'),
):
    content = await file.read()
    if model_key != captioning_service.model_key:
        captioning_service.load_captioning_model(model_key)
    try:
        result = captioning_service.generate_caption(
            content, conditional_text=conditional_text,
            max_new_tokens=max_new_tokens, num_beams=num_beams,
            num_captions=num_captions,
        )
    except Exception as e:
        raise HTTPException(500, f'캡셔닝 오류: {e}')
    return CaptionResponse(filename=file.filename, **result)

@router.post('/vqa', response_model=VQAResponse,
              summary='이미지 질의응답 (VQA)')
async def visual_qa(
    file: UploadFile = File(...),
    question: str = Form(..., description='이미지에 대한 질문 (영어 권장)'),
):
    content = await file.read()
    if captioning_service.vqa_model is None:
        raise HTTPException(503, 'VQA 모델 미로드')
    try:
        result = captioning_service.answer_question(content, question)
    except Exception as e:
        raise HTTPException(500, f'VQA 오류: {e}')
    return VQAResponse(filename=file.filename, **result)

```

이미지 캡셔닝 테스트

```

$ curl -X POST http://localhost:8000/caption/generate ₩
-F 'file=@cat.jpg' -F 'num_captions=3' -F 'num_beams=5'

```

```
# 응답 예시:
```

```
{  
  "primary_caption": "a cat sitting on a couch next to a window",  
  "captions": [  
    "a cat sitting on a couch next to a window",  
    "a gray cat is laying on a white couch",  
    "a cat resting on a sofa in a living room"  
  ],  
  "inference_time_sec": 1.234  
}
```

```
# VQA 테스트
```

```
$ curl -X POST http://localhost:8000/caption/vqa ₩  
  -F 'file=@cat.jpg' -F 'question=What color is the cat?'  
# → {"answer": "gray", ...}
```

6.4 UI 구현

이 절에서는 지금까지 구현한 세 가지 CV 서비스(이미지 분류, 객체 탐지, 이미지 캡셔닝)를 하나의 웹 UI로 통합합니다. Jinja2 템플릿 엔진과 FastAPI의 StaticFiles 미들웨어를 활용하여 사용자가 웹 브라우저에서 직접 이미지를 업로드하고 AI 결과를 시각적으로 확인할 수 있는 완전한 웹 애플리케이션을 완성합니다.

6.4.1 Jinja2 템플릿 구성

Jinja2는 Python 생태계에서 가장 널리 사용되는 HTML 템플릿 엔진입니다. FastAPI는 Jinja2Templates 클래스를 통해 HTML 템플릿 렌더링을 지원하며, 정적 파일(CSS, JS)은 StaticFiles 미들웨어로 서빙합니다.

Jinja2 주요 문법	설명 및 예시
{{ 변수명 }}	Python 변수 값 출력. {{ title }}, {{ result.score }}
{% if 조건 %} ... {% endif %}	조건부 렌더링.
{% for item in items %}	반복문. 리스트 순회.
{% extends 'base.html' %}	템플릿 상속. 공통 레이아웃 재사용.
{% block content %}...{% endblock %}	블록 정의. 자식 템플릿에서 내용 채움.
{{ url_for('static', path='css/style.css') }}	정적 파일 URL 생성.

main.py - Jinja2 및 StaticFiles 설정

```
# main.py (최종 통합)
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from fastapi.middleware.cors import CORSMiddleware
from routers import classification, detection, captioning, ui
from services.classification_service import classification_service
```

```
from services.detection_service import detection_service
from services.captioning_service import captioning_service

templates = Jinja2Templates(directory='templates')

@asynccontextmanager
async def lifespan(app: FastAPI):
    print('[STARTUP] Loading CV models...')
    classification_service.load_model()
    detection_service.load_model()
    captioning_service.load_captioning_model()
    captioning_service.load_vqa_model()
    print('[STARTUP] All models loaded!')
    yield

app = FastAPI(
    title='CV AI Web Services',
    description='컴퓨터 비전 AI 웹 서비스 (이미지 분류/객체 탐지/캡셔닝)',
    version='1.0.0',
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware, allow_origins=['*'],
    allow_methods=['*'], allow_headers=['*'],
)

# 정적 파일 서빙
app.mount('/static', StaticFiles(directory='static'), name='static')

# API 라우터 등록
app.include_router(classification.router)
app.include_router(detection.router)
app.include_router(captioning.router)

# UI 라우터 등록
```

```

app.include_router(ui.router)

@app.get('/api/health', tags=['Health'])
async def health():
    return {
        'classification': classification_service.pipe is not None,
        'detection': detection_service.model is not None,
        'captioning': captioning_service.caption_model is not None,
        'vqa': captioning_service.vqa_model is not None,
    }

```

6.4.2 템플릿 제작

templates/base.html - 공통 레이아웃

```

<!-- templates/base.html -->
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}CV AI Services{% endblock %}</title>
    <style>
        * { box-sizing: border-box; margin: 0; padding: 0; }
        body { font-family: 'Noto Sans KR', sans-serif;
            background: #f0f4f8; color: #1c1c1c; }
        nav { background: #1a3a6b; padding: 1rem 2rem;
            display: flex; align-items: center; gap: 2rem; }
        nav a { color: #fff; text-decoration: none; font-weight: 600; }
        nav a:hover { color: #a8d4f5; }
        nav .brand { font-size: 1.3rem; color: #a8d4f5; margin-right: auto; }
        .container { max-width: 960px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: #fff; border-radius: 12px;
            box-shadow: 0 2px 12px rgba(0,0,0,0.08);
            padding: 2rem; margin-bottom: 1.5rem; }
        h1 { color: #1a3a6b; font-size: 1.8rem; margin-bottom: 1rem; }
        h2 { color: #1f5c99; margin-bottom: 0.8rem; }
        .btn { background: #1a3a6b; color: #fff; border: none;

```

```

padding: 0.75rem 1.5rem; border-radius: 8px;
cursor: pointer; font-size: 1rem; }
.btn:hover { background: #2874a6; }
.alert-info { background:#eaf2fb; border:1px solid #2874a6;
padding:1rem; border-radius:8px; margin:1rem 0; }
.alert-error { background:#fddedec; border:1px solid #c0392b;
padding:1rem; border-radius:8px; margin:1rem 0; }
label { font-weight: 600; display: block; margin-bottom: 0.3rem; }
input[type=file], select, input[type=number], input[type=text] {
width: 100%; padding: 0.5rem; border: 1px solid #ccc;
border-radius: 6px; font-size: 0.95rem; }
.grid-2 { display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; }
.grid-3 { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 1rem; }
.progress-bar { background:#e0e0e0; border-radius:4px;
height:8px; margin-top:4px; }
.progress-fill { background:#1a3a6b; height:8px; border-radius:4px; }
</style>
</head>
<body>
  <nav>
    <span class="brand">CV AI Services</span>
    <a href="/">홈</a>
    <a href="/ui/classify">이미지 분류</a>
    <a href="/ui/detect">객체 탐지</a>
    <a href="/ui/caption">이미지 설명</a>
    <a href="/docs" style="color:#aee">API Docs</a>
  </nav>
  <div class="container">
    {% block content %}{% endblock %}
  </div>
  {% block scripts %}{% endblock %}
</body>
</html>

```

templates/index.html - 메인 페이지

```

<!-- templates/index.html -->
{% extends 'base.html' %}

```

```
{% block title %}CV AI Services - 메인{% endblock %}
```

```
{% block content %}
```

```
<div class="card" style="text-align:center; padding:3rem;">
```

```
<h1>컴퓨터 비전 AI 서비스</h1>
```

```
<p style="color:#666; margin:1rem 0 2rem;">
```

```
Hugging Face 모델 기반 이미지 분석 웹 서비스</p>
```

```
<div class="grid-3">
```

```
<a href="/ui/classify" style="text-decoration:none;">
```

```
<div class="card" style="border-top:4px solid #1a3a6b;">
```

```
<div style="font-size:3rem;">🔍</div>
```

```
<h2>이미지 분류</h2>
```

```
<p style="color:#666;">ViT / ResNet 기반<br>1,000개 클래스 분류</p>
```

```
</div>
```

```
</a>
```

```
<a href="/ui/detect" style="text-decoration:none;">
```

```
<div class="card" style="border-top:4px solid #27ae60;">
```

```
<div style="font-size:3rem;">🖼️</div>
```

```
<h2>객체 탐지</h2>
```

```
<p style="color:#666;">YOLOv8 기반<br>80개 COCO 클래스 탐지</p>
```

```
</div>
```

```
</a>
```

```
<a href="/ui/caption" style="text-decoration:none;">
```

```
<div class="card" style="border-top:4px solid #8e44ad;">
```

```
<div style="font-size:3rem;">💬</div>
```

```
<h2>이미지 설명</h2>
```

```
<p style="color:#666;">BLIP 기반<br>이미지를 자연어로 설명</p>
```

```
</div>
```

```
</a>
```

```
</div>
```

```
</div>
```

```
<div class="card">
```

```
<h2>API 문서</h2>
```

```
<p style="margin-bottom:1rem;">모든 API는 Swagger UI에서 직접 테스트할 수 있습니다.</p>
```

```
<div style="display:flex; gap:1rem;">
```

```
<a href="/docs" class="btn">Swagger UI</a>
```

```
<a href="/redoc" class="btn" style="background:#6c757d;">ReDoc</a>
```

```
</div>
```

```
</div>
```



```
{% endblock %}
```

6.4.3 템플릿 구현

templates/classify.html - 이미지 분류 UI

```
<!-- templates/classify.html -->
{% extends 'base.html' %}
{% block title %}이미지 분류{% endblock %}
{% block content %}
<div class="card">
  <h1>🔍 이미지 분류 (Image Classification)</h1>
  <p>ViT, ResNet 등 딥러닝 모델로 이미지의 카테고리를 분류합니다.</p>
  <form id="classifyForm" style="margin-top:1.5rem;">
    <div style="margin-bottom:1rem;">
      <label>이미지 파일 선택</label>
      <input type="file" id="imageFile" accept="image/*" required>
    </div>
    <div class="grid-2" style="margin-bottom:1rem;">
      <div>
        <label>모델 선택</label>
        <select id="modelKey">
          <option value="vit-base">ViT-Base (권장)</option>
          <option value="resnet-50">ResNet-50 (빠름)</option>
          <option value="convnext-base">ConvNeXt-Base (고품질)</option>
          <option value="efficientnet">EfficientNet-B4</option>
        </select>
      </div>
      <div>
        <label>Top-K 결과 수</label>
        <input type="number" id="topK" value="5" min="1" max="20">
      </div>
    </div>
    <div style="display:flex; gap:1rem; align-items:center;">
      <button type="submit" class="btn">분류하기</button>
      <span id="loading" style="display:none;color:#666;">분류 중...</span>
    </div>
  </form>
```

```

</div>
<div id="previewSection" style="display:none;">
  <div class="card">
    <h2>업로드된 이미지</h2>
    <img id="previewImg" style="max-width:100%; max-height:400px;
      border-radius:8px;">
  </div>
</div>
<div id="resultSection" style="display:none;">
  <div class="card">
    <h2>분류 결과</h2>
    <div id="topResult" class="alert-info"></div>
    <div id="predBars" style="margin-top:1rem;"></div>
    <p id="metaInfo" style="margin-top:1rem;color:#666;font-size:0.9rem;"></p>
  </div>
</div>
<div id="errorSection" class="alert-error" style="display:none;"></div>
{% endblock %}
{% block scripts %}
<script>
document.getElementById('imageFile').addEventListener('change', e => {
  const file = e.target.files[0];
  if (!file) return;
  const reader = new FileReader();
  reader.onload = ev => {
    document.getElementById('previewImg').src = ev.target.result;
    document.getElementById('previewSection').style.display = 'block';
  };
  reader.readAsDataURL(file);
});
document.getElementById('classifyForm').addEventListener('submit', async e => {
  e.preventDefault();
  const loading = document.getElementById('loading');
  const resultSection = document.getElementById('resultSection');
  const errorSection = document.getElementById('errorSection');
  loading.style.display = 'inline';
  resultSection.style.display = 'none';
  errorSection.style.display = 'none';

```

```

const fd = new FormData();
fd.append('file', document.getElementById('imageFile').files[0]);
fd.append('model_key', document.getElementById('modelKey').value);
fd.append('top_k', document.getElementById('topK').value);
try {
  const resp = await fetch('/classify/upload', { method:'POST', body:fd });
  loading.style.display = 'none';
  if (!resp.ok) {
    const err = await resp.json();
    errorSection.textContent = '오류: ' + JSON.stringify(err.detail);
    errorSection.style.display = 'block'; return;
  }
  const data = await resp.json();
  document.getElementById('topResult').innerHTML =
    `<strong>최고 예측: ${data.top_prediction}</strong>` +
    ` (신뢰도: ${(data.top_score*100).toFixed(1)}%)`;
  let html = "";
  data.predictions.forEach(p => {
    const pct = (p.score*100).toFixed(2);
    html += `<div style="margin-bottom:0.6rem;">
      <div style="display:flex;justify-content:space-between;">
        <span>${p.rank}. ${p.label}</span> <span>${pct}%</span>
      </div>
      <div class="progress-bar">
        <div class="progress-fill" style="width:${pct}%"></div>
      </div></div>`;
  });
  document.getElementById('predBars').innerHTML = html;
  document.getElementById('metaInfo').textContent =
    `모델: ${data.model_used} | 추론: ${data.inference_time_sec}s | 크기: ${data.image_size}`;
  resultSection.style.display = 'block';
} catch(err) {
  loading.style.display = 'none';
  errorSection.textContent = '오류: ' + err.message;
  errorSection.style.display = 'block';
}
});
</script>

```

```
{% endblock %}
```

templates/detect.html - 객체 탐지 UI

```
<!-- templates/detect.html -->
{% extends 'base.html' %}
{% block title %}객체 탐지{% endblock %}
{% block content %}
<div class="card">
  <h1>🖼️ 객체 탐지 (Object Detection)</h1>
  <p>YOLOv8 모델로 이미지 속 객체의 위치와 종류를 탐지합니다.</p>
  <form id="detectForm" style="margin-top:1.5rem;">
    <div style="margin-bottom:1rem;">
      <label>이미지 파일</label>
      <input type="file" id="detectFile" accept="image/*" required>
    </div>
    <div class="grid-3" style="margin-bottom:1rem;">
      <div>
        <label>모델 선택</label>
        <select id="yoloModel">
          <option value="yolov8n">YOLOv8n (빠름)</option>
          <option value="yolov8s">YOLOv8s (균형)</option>
          <option value="yolov8m" selected>YOLOv8m (권장)</option>
        </select>
      </div>
      <div>
        <label>신뢰도 임계값: <span id="confLabel">0.40</span></label>
        <input type="range" id="confSlider" min="0.1" max="0.9"
          step="0.05" value="0.4"
          style="margin-top:0.5rem;">
      </div>
      <div>
        <label>응답 형식</label>
        <select id="respType">
          <option value="json">JSON (결과 목록)</option>
          <option value="image">시각화 이미지</option>
        </select>
      </div>
    </div>
  </div>
</div>
```

```

    </div>
    <button type="submit" class="btn">탐지하기 </button>
    <span id="detectLoading" style="display:none;"> 탐지 중...</span>
  </form>
</div>

<div id="detectPreview" style="display:none;">
  <div class="card">
    <h2>결과 이미지</h2>
    <img id="resultImg" style="max-width:100%; border-radius:8px;">
    <div id="detectSummary" class="alert-info" style="margin-top:1rem;"> </div>
  </div>
</div>

<div id="detectJsonResult" style="display:none;">
  <div class="card">
    <h2>탐지 결과</h2>
    <div id="detectSummaryJson" class="alert-info"> </div>
    <div id="detectTable" style="overflow-x:auto; margin-top:1rem;"> </div>
  </div>
</div>
{% endblock %}

{% block scripts %}
<script>
document.getElementById('confSlider').addEventListener('input', e => {
  document.getElementById('confLabel').textContent =
    parseFloat(e.target.value).toFixed(2);
});

document.getElementById('detectForm').addEventListener('submit', async e => {
  e.preventDefault();
  const loading = document.getElementById('detectLoading');
  loading.style.display = 'inline';
  document.getElementById('detectPreview').style.display = 'none';
  document.getElementById('detectJsonResult').style.display = 'none';
}

```

```

const conf = document.getElementById('confSlider').value;
const respType = document.getElementById('respType').value;
const file = document.getElementById('detectFile').files[0];
const fd = new FormData();
fd.append('file', file);
fd.append('confidence', conf);

try {
  if (respType === 'image') {
    const resp = await fetch('/detect/visualize', { method:'POST', body:fd });
    loading.style.display = 'none';
    if (!resp.ok) { alert('탐지 오류'); return; }
    const blob = await resp.blob();
    const url = URL.createObjectURL(blob);
    document.getElementById('resultImg').src = url;
    const total = resp.headers.get('X-Total-Objects') || '?';
    const time = resp.headers.get('X-Inference-Time') || '?';
    document.getElementById('detectSummary').textContent =
      `탐지 객체 수: ${total}개 | 추론 시간: ${time}초`;
    document.getElementById('detectPreview').style.display = 'block';
  } else {
    fd.append('include_visualization', 'true');
    const resp = await fetch('/detect/json', { method:'POST', body:fd });
    loading.style.display = 'none';
    const data = await resp.json();
    const counts = Object.entries(data.class_counts)
      .map(([k,v]) => `${k}: ${v}개`).join(', ');
    document.getElementById('detectSummaryJson').innerHTML =
      `<strong>총 ${data.total_objects}개 객체 탐지</strong>` +
      (counts ? ` | ${counts}` : '');
    if (data.visualization_base64) {
      document.getElementById('resultImg').src = data.visualization_base64;
      document.getElementById('detectPreview').style.display = 'block';
    }

    let tableHtml = '<table style="width:100%;border-collapse:collapse;">';
    tableHtml += '<tr style="background:#1a3a6b;color:#fff;">';
    tableHtml += '<th style="padding:8px;">ID</th><th>클래스</th>';
    tableHtml += '<th>신뢰도</th><th>위치 (x1,y1,x2,y2)</th></tr>';
  }
}

```

```

data.detections.forEach((d,i) => {
  const bg = i%2===0 ? '#eaf2fb' : '#fff';
  const b = d.bbox;
  tableHtml += `<tr style="background:${bg};">
    <td style="padding:8px;text-align:center;">${d.id}</td>
    <td style="padding:8px;">${d.class_name}</td>
    <td style="padding:8px;text-align:center;">
      ${((d.confidence*100).toFixed(1))}%</td>
    <td style="padding:8px;font-family:monospace;">
      (${b.x1},${b.y1}) - (${b.x2},${b.y2})</td></tr>`;
});
tableHtml += '</table>';
document.getElementById('detectTable').innerHTML = tableHtml;
document.getElementById('detectJsonResult').style.display = 'block';
}
} catch(err) {
  loading.style.display = 'none';
  alert('오류: ' + err.message);
}
});
</script>
{% endblock %}

```

templates/caption.html - 이미지 캡셔닝 UI

```

<!-- templates/caption.html -->
{% extends 'base.html' %}
{% block title %}이미지 설명 생성{% endblock %}
{% block content %}
<div class="card">
  <h1>🗨 이미지 설명 생성 (Image Captioning)</h1>
  <p>BLIP 모델이 이미지를 보고 자연어로 설명합니다.</p>
  <div style="margin-top:1rem;">
    <button id="tabCaption" class="btn" onclick="showTab('caption')">캡션 생성</button>
    <button id="tabVqa" class="btn" style="background:#8e44ad;margin-left:0.5rem;"
      onclick="showTab('vqa')">이미지 Q&A</button>
  </div>
</div>
</div>

```

```

<!-- 캡션 생성 탭 -->
<div id="captionTab">
  <div class="card">
    <h2>캡션 생성</h2>
    <form id="captionForm">
      <div style="margin-bottom:1rem;">
        <label>이미지 파일</label>
        <input type="file" id="captionFile" accept="image/*" required>
      </div>
      <div class="grid-2" style="margin-bottom:1rem;">
        <div>
          <label>유도 텍스트 (선택)</label>
          <input type="text" id="condText"
            placeholder="예: a photo of">
        </div>
        <div>
          <label>캡션 개수</label>
          <input type="number" id="numCaptions" value="3" min="1" max="5">
        </div>
      </div>
      <button type="submit" class="btn">설명 생성</button>
      <span id="captionLoading" style="display:none;"> 생성 중...</span>
    </form>
  </div>
  <div id="captionResult" style="display:none;">
    <div class="card">
      <h2>생성된 캡션</h2>
      <div id="captionPreview"></div>
      <div id="captionList" style="margin-top:1rem;"></div>
      <p id="captionMeta" style="margin-top:1rem;color:#666;font-size:0.9rem;"></p>
    </div>
  </div>
</div>

<!-- VQA 탭 -->
<div id="vqaTab" style="display:none;">
  <div class="card">

```



```

<h2>이미지 Q&A (Visual Question Answering)</h2>
<form id="vqaForm">
  <div style="margin-bottom:1rem;">
    <label>이미지 파일</label>
    <input type="file" id="vqaFile" accept="image/*" required>
  </div>
  <div style="margin-bottom:1rem;">
    <label>질문 (영어 권장)</label>
    <input type="text" id="vqaQuestion"
      placeholder="예: What color is the cat? / How many people?"
      required>
  </div>
  <button type="submit" class="btn" style="background:#8e44ad;">
    질문하기</button>
  <span id="vqaLoading" style="display:none;"> 답변 생성 중...</span>
</form>
</div>
<div id="vqaResult" style="display:none;">
  <div class="card">
    <h2>AI 답변</h2>
    <div id="vqaAnswer" class="alert-info"></div>
  </div>
</div>
</div>
{% endblock %}

{% block scripts %}
<script>
function showTab(tab) {
  document.getElementById('captionTab').style.display = tab==='caption' ? 'block' : 'none';
  document.getElementById('vqaTab').style.display = tab==='vqa' ? 'block' : 'none';
}

document.getElementById('captionForm').addEventListener('submit', async e => {
  e.preventDefault();
  const loading = document.getElementById('captionLoading');
  loading.style.display = 'inline';
  const fd = new FormData();

```

```

fd.append('file', document.getElementById('captionFile').files[0]);
const ct = document.getElementById('condText').value;
if (ct) fd.append('conditional_text', ct);
fd.append('num_captions', document.getElementById('numCaptions').value);
fd.append('num_beams', document.getElementById('numCaptions').value);
try {
  const resp = await fetch('/caption/generate', { method:'POST', body:fd });
  loading.style.display = 'none';
  const data = await resp.json();
  const file = document.getElementById('captionFile').files[0];
  const reader = new FileReader();
  reader.onload = ev => {
    document.getElementById('captionPreview').innerHTML =
      ``;
  };
  reader.readAsDataURL(file);
  let html = "";
  data.captions.forEach((c, i) => {
    const bg = i===0 ? '#eaf2fb' : '#f8f9fa';
    const badge = i===0 ? '<span style="background:#1a3a6b;color:#fff;
      padding:2px 8px;border-radius:4px;font-size:0.8rem;margin-right:8px;">
      Best</span>' : '<span style="color:#666;margin-right:8px;">${i+1}</span>`;
    html += `<div style="background:${bg};padding:0.8rem;
      border-radius:8px;margin-bottom:0.5rem;">${badge}${c}</div>`;
  });
  document.getElementById('captionList').innerHTML = html;
  document.getElementById('captionMeta').textContent =
    `모델: ${data.model_used} | 추론: ${data.inference_time_sec}s`;
  document.getElementById('captionResult').style.display = 'block';
} catch(err) {
  loading.style.display = 'none'; alert('오류: '+err.message);
}
});

document.getElementById('vqaForm').addEventListener('submit', async e => {
  e.preventDefault();
  const loading = document.getElementById('vqaLoading');

```

```

loading.style.display = 'inline';
const fd = new FormData();
fd.append('file', document.getElementById('vqaFile').files[0]);
fd.append('question', document.getElementById('vqaQuestion').value);
try {
    const resp = await fetch('/caption/vqa', { method:'POST', body:fd });
    loading.style.display = 'none';
    const data = await resp.json();
    document.getElementById('vqaAnswer').innerHTML =
        `<strong>질문:</strong> ${data.question}<br><br>` +
        `<strong style="color:#1a3a6b;">AI 답변:</strong> ${data.answer}`;
    document.getElementById('vqaResult').style.display = 'block';
} catch(err) {
    loading.style.display = 'none'; alert('오류: '+err.message);
}
});
</script>
{% endblock %}

```

6.4.4 컴퓨터 비전 웹 서비스 UI 완성

routes/ui.py - UI 라우터

```

# routes/ui.py
from fastapi import APIRouter, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates

router = APIRouter(prefix="", tags=['Web UI'])
templates = Jinja2Templates(directory='templates')

@router.get('/', response_class=HTMLResponse, summary='메인 페이지')
async def index(request: Request):
    return templates.TemplateResponse(
        'index.html', {'request': request}
    )

@router.get('/ui/classify', response_class=HTMLResponse,

```

```

summary='이미지 분류 UI')
async def classify_ui(request: Request):
    return templates.TemplateResponse(
        'classify.html', {'request': request}
    )

@router.get('/ui/detect', response_class=HTMLResponse,
            summary='객체 탐지 UI')
async def detect_ui(request: Request):
    return templates.TemplateResponse(
        'detect.html', {'request': request}
    )

@router.get('/ui/caption', response_class=HTMLResponse,
            summary='이미지 캡셔닝 UI')
async def caption_ui(request: Request):
    return templates.TemplateResponse(
        'caption.html', {'request': request}
    )

```

6.4.5 UI 최종 테스트 및 정리

모든 서비스와 UI가 완성되었습니다. 전체 서버를 실행하고 브라우저에서 각 페이지를 테스트합니다.

```

# 서버 실행
$ uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# 브라우저에서 접속:
# http://localhost:8000/      → 메인 페이지 (서비스 선택)
# http://localhost:8000/ui/classify → 이미지 분류 UI
# http://localhost:8000/ui/detect  → 객체 탐지 UI
# http://localhost:8000/ui/caption → 이미지 캡셔닝 UI
# http://localhost:8000/docs      → Swagger API 문서

# API 전체 엔드포인트:

```

# POST /classify/upload	이미지 파일 분류
# POST /classify/base64	Base64 이미지 분류
# POST /classify/url	URL 이미지 분류
# GET /classify/models	지원 모델 목록
#	
# POST /detect/json	객체 탐지 JSON 결과
# POST /detect/visualize	객체 탐지 시각화 이미지
# GET /detect/classes	탐지 가능한 클래스 목록
#	
# POST /caption/generate	이미지 캡션 생성
# POST /caption/vqa	이미지 질의응답
#	
# GET /	메인 UI
# GET /ui/classify	분류 UI
# GET /ui/detect	탐지 UI
# GET /ui/caption	캡셔닝 UI

UI 페이지	기능 요약
메인 (/)	세 서비스 카드 링크, Swagger/ReDoc 바로가기
/ui/classify	이미지 파일 선택 → 모델 선택 → 분류 결과 바 차트 표시
/ui/detect	이미지 파일 선택 → 신뢰도 슬라이더 → JSON 표 또는 시각화 이미지
/ui/caption	캡션 생성 탭 + VQA 탭. 캡션 Best 배지 표시, 질문-답변 표시

🔗 6장 학습 정리

이미지 분류의 원리와 ResNet, ViT, ConvNeXt 모델 구조를 비교하고 Pipeline API로 구현했습니다.

HF pipeline task= 'image-classification'으로 전처리~ 후처리를 자동화했습니다.

Base64 인코딩 이미지를 JSON으로 받는 방법과 URL 이미지 분류를 구현했습니다.

YOLOv8 5가지 크기 모델을 비교하고, 바운딩 박스 탐지 + PIL 시각화를 완성했습니다.

JSON 결과와 시각화 이미지 두 가지 응답 방식을 분리하여 API를 설계했습니다.

BLIP의 조건부/무조건부 캡션 생성과 VQA 기능을
BlipProcessor/BlipForConditionalGeneration으로 구현했습니다.

Jinja2 + StaticFiles로 세 서비스를 통합하는 완전한 웹 UI를 구축했습니다.

JavaScript Fetch API로 비동기 이미지 업로드와 실시간 결과 표시를 구현했습니다.

연습 문제

7. Hugging Face pipeline task= 'image-classification'이 자동으로 처리해주는 전처리 단계를 3가지 이상 나열하고, 각각의 역할을 설명하세요.
8. YOLOv8에서 confidence 임계값(threshold)과 IoU 임계값(iou_threshold)의 역할을 각각 설명하고, 두 값을 낮추거나 높였을 때 탐지 결과에 미치는 영향을 비교하세요.
9. BLIP의 '조건부 캡션 생성'과 '무조건부 캡션 생성'의 차이를 설명하고, 각각 언제 사용하는 것이 적합한지 예시를 들어 설명하세요.
10. 다음 FastAPI 코드에서 UploadFile과 일반 JSON Body를 동시에 사용할 수 없는 이유를 설명하고, 해결 방법 2가지를 제시하세요.

잘못된 코드:

```
@router.post('/upload')
```

```
async def upload(file: UploadFile = File(...), data: MyModel = Body(...)):
```

```
...
```

11. Jinja2 템플릿의 템플릿 상속(extends) 패턴을 사용하는 이유를 설명하고, base.html에서 {% block content %}를 정의하지 않으면 어떤 문제가 발생하는지 서술하세요.
12. [코딩 실습] POST /classify/batch 엔드포인트를 구현하세요. 여러 이미지를 동시에 업로드하면 각각의 분류 결과를 리스트로 반환합니다. (힌트: List[UploadFile] 활용)

6장 요약

이 장에서는 컴퓨터 비전 AI 기술을 FastAPI와 결합하여 4가지 핵심 서비스를 구현했습니다.

- 이미지 분류 API: HuggingFace pipeline으로 ViT, ResNet 등 5가지 모델을 지원하는 분류 서비스. UploadFile, Base64, URL 3가지 입력 방식 모두 지원.
- 객체 탐지 API: ultralytics YOLOv8로 COCO 80개 클래스 탐지. JSON 결과 반환과 PIL 기반 시각화 이미지 반환 두 가지 엔드포인트 구현.
- 이미지 캡셔닝 API: BLIP 모델로 조건부/무조건부 캡션 생성과 VQA 기능 구현. BlipProcessor와 BlipForConditionalGeneration 심층 활용.
- 통합 웹 UI: Jinja2 템플릿 상속 구조로 base.html → 각 서비스 페이지. Fetch API 비동기 처리, 진행 표시, 결과 시각화까지 완성.

□ 다음 장 예고: 제7장 RAG(검색 증강 생성) 시스템 구축

7장에서는 LLM과 벡터 DB를 결합하여 문서 기반 AI 질의응답 시스템(RAG)을 구축합니다.

주요 내용:

- RAG(Retrieval-Augmented Generation) 아키텍처 이해
- PDF/Word 문서 청킹(Chunking) 및 임베딩
- ChromaDB / FAISS 벡터 저장소 구축
- LLM과 검색 결과 결합한 답변 생성
- FastAPI 기반 문서 업로드 및 질의응답 API
- 대화 이력 유지 멀티턴 RAG 구현